

Contents

Stripe production setup – partner playbook	1
1. Activate the account	1
2. Make sure live API keys are in Vercel	1
3. Set up the webhook endpoint (live mode)	2
4. Enable payment methods beyond cards	2
5. Verify the Apple Pay domain	2
6. Configure the Customer Portal	3
7. Set the refund policy on the Stripe products	3
8. Set up Stripe Tax (only if/when you register for sales tax)	4
9. Set up Stripe email branding	4
10. Smart Retries (dunning) – already on by default	4
11. Test the full flow end-to-end	4
Environment variables summary	5
Troubleshooting card declines (Radar)	5
What I (the code) already handle	6

Stripe production setup – partner playbook

Everything that has to happen in the Stripe Dashboard to make Brikk’s payment flow production-ready. Code is already wired (`app/api/stripe/route.js`, `app/api/stripe/webhook/route.js`, `app/api/stripe/portal/route.js`) – these steps activate the features.

Work top-down. Each step has a verification at the end.

1. Activate the account

Stripe accounts are in “test mode” by default and reject real cards. To accept real money:

1. Stripe Dashboard -> top-left, click the toggle that says **Test mode** -> switch to **Live mode**.
2. If the account is still in onboarding, Stripe will show a yellow banner: “Activate payments.” Click it.
3. Provide:
 - Legal business name
 - EIN (or SSN if sole proprietor)
 - Business address
 - Bank account for payouts
 - SSN of the business owner (for KYC)
4. Stripe reviews this within a few hours, sometimes immediately.

Verified when: the yellow “Activate payments” banner disappears and you can see “Charges enabled: Yes” in Stripe -> Settings -> Account details.

2. Make sure live API keys are in Vercel

Once the account is live, the test-mode keys (`sk_test_...`, `pk_test_...`) won’t work for real payments.

1. Stripe -> Developers -> **API keys** -> switch to **Live mode** if not already.
2. Copy the **Secret key** (`sk_live_...`).
3. Vercel -> Brikk project -> Settings -> Environment Variables -> update `STRIPE_SECRET_KEY` to the live key. Both Production and Preview environments.
4. Redeploy.

Verified when: a fresh test checkout from `/app/upgrade` succeeds with a real card.

3. Set up the webhook endpoint (live mode)

The webhook handler at `/api/stripe/webhook` already exists, but Stripe needs to be told to send events to it.

1. Stripe -> Developers -> Webhooks -> + **Add endpoint** (in live mode!)
2. **Endpoint URL:** `https://brikk.store/api/stripe/webhook`
3. **Description:** Brikk production webhook
4. **Events to send** – select these:
 - `checkout.session.completed`
 - `customer.subscription.created`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `customer.subscription.paused`
 - `customer.subscription.resumed`
 - `customer.subscription.trial_will_end`
 - `invoice.payment_failed`
5. Click **Add endpoint**.
6. On the next screen, click **Reveal** next to “Signing secret” and copy it (starts with `whsec_`).
7. Vercel -> Environment Variables -> set `STRIPE_WEBHOOK_SECRET` to that value. Production + Preview.
8. Redeploy.

Verified when: Stripe -> Developers -> Webhooks -> click your endpoint -> “Recent attempts” shows 200 OK responses after a test event.

To send a test event: click ... on the endpoint -> “Send test webhook” -> pick `checkout.session.completed` -> Send. Should land 200.

4. Enable payment methods beyond cards

The code no longer hardcodes “card only” – it reads from your Dashboard’s Payment Methods settings. So whatever you toggle on here, customers see.

1. Stripe -> Settings -> Payment methods.
2. Recommended for Brikk:
 - **Cards (Visa, Mastercard, Amex, Discover)** – required, on by default.
 - **Apple Pay / Google Pay** – high mobile conversion. Requires domain verification (next step).
 - **Link** – Stripe’s one-click checkout. Adds zero friction, recommended.
 - **Cash App Pay** – useful for younger US customers.
3. Skip these for now (not relevant to real-estate SaaS):
 - SEPA Direct Debit, Bancontact, iDEAL, BLIK – European bank methods, no US use case for you.
 - ACH Direct Debit – useful for B2B but adds a 3-5 day settlement delay. Add later if you want.
4. Save.

Verified when: Settings -> Payment methods shows green [x] next to each method you enabled.

5. Verify the Apple Pay domain

Apple won’t show the Apple Pay button on iOS Safari unless `brikk.store` proves ownership.

The code already serves the verification file at `/.well-known/apple-developer-merchantid-domain-association`. You just need to put the Stripe-generated content into the env var.

1. Stripe -> Settings -> Payment methods -> Apple Pay -> **Add a new domain**.
2. Enter **brikk.store** (no https://, no path).
3. Stripe shows a **Download verification file** button. Click it. You get a file or a long text string.
4. Open the file in a text editor and copy the entire content (it's one long string, no line breaks).
5. Vercel -> Environment Variables -> **Add New**:
 - Key: `APPLE_PAY_DOMAIN_ASSOCIATION`
 - Value: paste the entire string
 - Environment: Production
6. Redeploy.
7. Back in Stripe Dashboard, click **Verify** next to the domain.

Verified when: Stripe shows “Verified” next to **brikk.store** under Apple Pay domains. Test by opening **brikk.store** on an iPhone in Safari and starting a subscription – Apple Pay button should appear.

6. Configure the Customer Portal

The portal is where subscribed users update their card, cancel, download invoices. The code at `/api/stripe/portal/route.js` opens it; you have to enable and configure it once.

1. Stripe -> Settings -> Billing -> **Customer portal**.
2. Toggle **Customer portal** ON.
3. Branding section:
 - **Business name:** Brikk
 - **Logo:** upload `public/icon-512.png` (or any version of the brick logo)
 - **Colors:** accent `#1A1A18` (your text color)
4. Functionality section – enable:
 - [OK] Customers can update their payment method
 - [OK] Customers can view their invoice history
 - [OK] Customers can update their email and address
 - [OK] Customers can cancel subscriptions
 - Cancellation policy: **At end of billing period** (don't refund prorated time)
 - Cancellation reason: Optional, helpful for churn analysis
 - ? Customers can switch plans – disable unless you want users to self-downgrade. Better to have them email you.
5. Save changes.

Verified when: subscribe with a test account, go to `/app/settings` -> Billing tab, click “Open billing portal” – you land in the Stripe-hosted portal showing your branding.

7. Set the refund policy on the Stripe products

This makes the “no refunds” policy visible to anyone reading invoices and gives you a stronger position with card-issuer disputes.

1. Stripe -> Products -> click each Brikk product (Pro Monthly, Pro Setup, Team Monthly, Team Setup).
2. In the **Metadata** section, add:
 - Key: `refund_policy`
 - Value: `All sales final. No refunds. See https://brikk.store/terms`
3. Save.

You can also configure the public-facing receipts:

1. Stripe -> Settings -> Email – enable “Send finalized invoices” and “Send credit notes.”

2. In **Receipt settings**, add a footer: All sales are final. Cancel anytime to stop future charges. Full terms at <https://brikk.store/terms>.
-

8. Set up Stripe Tax (only if/when you register for sales tax)

Skip this until you've decided to register for sales tax somewhere (California is the most likely first state since you're based in SoCal).

When you're ready:

1. Stripe -> Tax -> **Registrations** -> add the states/countries where you've registered.
2. Stripe -> Tax -> Settings -> set your origin address.
3. Add Vercel env var `STRIPE_ENABLE_TAX=true` and redeploy. (This flips the code to pass `automatic_tax[enabled]=true` to checkout.)

Don't enable tax until registrations are in place – Stripe will error every checkout if it can't compute tax for the buyer's location.

9. Set up Stripe email branding

The receipts and trial-ending emails Stripe sends should look like Brikk emails, not generic Stripe.

1. Stripe -> Settings -> Branding.
2. **Icon**: upload `public/icon-512.png` (square brick logo).
3. **Logo**: upload `brand/brikk-wordmark-dark-3200.png` for the lighter customer-facing emails. (For dark-mode emails, also upload `brand/brikk-wordmark-white-3200.png`.)
4. **Brand color**: `#1A1A18`
5. **Accent color**: `#16803C` (your success green) – used for buttons.
6. Save.

Verified when: subscribe with a test account -> the receipt email has the Brikk logo and brand color.

10. Smart Retries (dunning) – already on by default

When a renewal payment fails, Stripe automatically retries the card 4 times over ~3 weeks. No action needed; just verify the setting:

1. Stripe -> Settings -> Billing -> **Subscription and emails**.
2. Confirm **Smart Retries** is enabled.
3. Confirm **Automatically email customers about failed payments** is enabled.

If a customer's card permanently fails, the subscription enters **past_due** (your webhook handles this), then **canceled** after the retry window. The user is sent to `/app/upgrade` by the trial-enforcement layer.

11. Test the full flow end-to-end

Before announcing launch:

1. In incognito, go to `brikk.store/login`, sign up with a throwaway email.
2. Log in. Land on `/app`. Sidebar should show "Trial – 14 days left."
3. Go to Settings -> Billing. Click "Subscribe to Pro."
4. Hosted Stripe checkout opens. Verify:
 - [OK] "Apple Pay" button shows on iOS Safari (after step 5)

- [OK] “Link” prefills if you’ve used Link before
 - [OK] Billing address required
 - [OK] 14-day trial mentioned
5. Use Stripe’s test card 4242 4242 4242 4242, any future expiry, any CVC.
 6. Complete checkout.
 7. Land back on `/app/settings?payment=success`.
 8. Within 1-2 seconds, the billing tab should show “Pro plan - Trialing.”
 9. Click “Open billing portal.” Should open Stripe-hosted portal.
 10. In Stripe Dashboard -> Customers -> find this customer -> simulate “Cancel subscription.”
 11. Check that `/app/settings` Billing tab now shows the plan picker (since subscription is canceled).

If all 11 work, you’re production-ready.

Environment variables summary

What needs to be in Vercel (Production):

Key	Value	Required
STRIPE_SECRET_KEY	sk_live_... from Stripe -> Developers -> Keys	YES
STRIPE_WEBHOOK_SECRET	whsec_... from step 3 above	YES
APPLE_PAY_DOMAIN_ASSOCIATION	The long string from step 5	For Apple Pay
STRIPE_ENABLE_TAX	true or unset	Only when tax registered
NEXT_PUBLIC_APP_URL	https://brikk.store	YES

Troubleshooting card declines (Radar)

Nathan tightened Radar fraud rules to block highest-risk transactions, fail on CVC mismatch, fail on postal code mismatch, and require 3DS challenge on supported cards. This catches fraud but will occasionally decline legitimate cards. Here’s how to recognize what’s happening:

Customer reports “my card was declined”:

1. Stripe Dashboard -> Payments -> find the failed charge.
2. Click into it. Look at the **Decline reason** field.
3. Match against this table:

Decline reason	What it means	What to tell the customer
card_declined (generic)	Issuer declined for unspecified reason	“Your bank rejected the charge. Try a different card or call your bank’s fraud line.”
incorrect_cvc	CVC didn’t match	“The 3- or 4-digit code on the back of the card didn’t match. Re-enter and try again.”
incorrect_zip / postal_code_invalid	Billing ZIP didn’t match card on file	“The billing ZIP code didn’t match your card’s address. Use your bank statement’s address.”
insufficient_funds	Card has insufficient funds	“Insufficient funds on the card. Try a different card.”
expired_card	Card past expiry	“The card has expired. Please use a current card.”
authentication_required	3DS challenge required but failed	“Your bank needs extra verification. Complete the prompt on your bank’s app or website.”

Decline reason	What it means	What to tell the customer
<code>do_not_honor</code>	Issuer blocked transaction	“Your bank blocked this charge. Call them – usually a fraud-prevention flag.”
Radar rule: <code>risk_level=highest</code>	Stripe Radar flagged as high fraud risk	“Stripe blocked this for security. Try Apple Pay or Link instead – they include extra verification that helps clear the flag.”

The code already shows friendly user messages for these via `friendlyStripeError()` in `app/api/stripe/route.js`. So most of the time the customer will see a clear explanation. This table is for when you need to investigate a specific case.

To temporarily loosen Radar for a known-good customer: - Stripe -> Radar -> Lists -> add their email to “allow_list” or their card fingerprint to “trusted_payment_methods”. - Their next attempt should clear regardless of risk level.

Never share decline-code details with the customer in front-end UI – it gives fraudsters too much info. The friendly mapping intentionally generalizes.

What I (the code) already handle

- [OK] Checkout session creation with 14-day trial, billing address required, tax ID collection, phone collection
- [OK] Required TOS agreement at checkout (`consent_collection`)
- [OK] Friendly user-facing error mapping for ~10 Stripe decline codes
- [OK] Trial-ending heads-up email 3 days before charge (reduces surprise-charge chargebacks)
- [OK] Settings-page success/cancel banner when returning from checkout or portal
- [OK] Webhook signature verification with replay protection
- [OK] Idempotent team creation on `checkout.session.completed`
- [OK] Subscription status sync on create/update/delete/pause/resume/trial_will_end/payment_failed
- [OK] Customer Portal session creation gated by Supabase auth
- [OK] Apple Pay domain verification file route (just needs the env var)
- [OK] Subscription-aware billing tab (different UI for trialing/active/past-due/never-subscribed)
- [OK] Hard paywall redirect to `/app/upgrade` for expired trials
- [OK] “All sales final” disclosure on pricing page, paywall page, billing tab, and TOS

What I don’t and can’t handle (your partner’s playbook above):

- ? Activating the live Stripe account
- ? Toggling Apple Pay / Link / Cash App in Dashboard
- ? Configuring the Customer Portal
- ? Email branding inside Stripe
- ? Sales tax registration